

CONCURRENCY, PROCESSES, THREADS, CORES, AND SCHEDULING – THE COMPLETE PICTURE

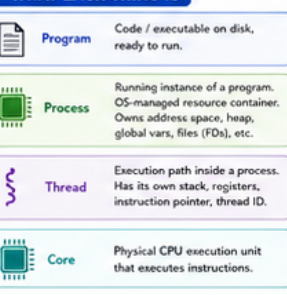
A detailed visual guide for C / Linux beginners

LEGEND: Program (Blue), Process (Green), Thread (Purple), Scheduler / Run Queue (Orange), CPU Core / Hardware (Teal), Shared Memory (Light Green), Private Thread State (Light Purple), Problems / Hazards (Red), Solutions / Synchronization (Light Green), Waiting / Blocked (Grey)

1 THE BIG PIPELINE

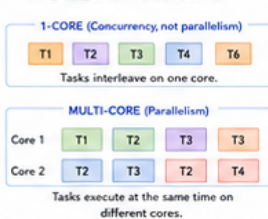


2 WHAT EACH THING IS

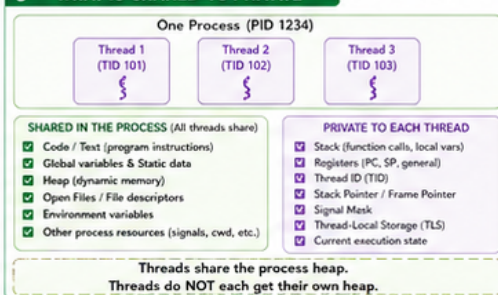


CONCURRENCY vs PARALLELISM

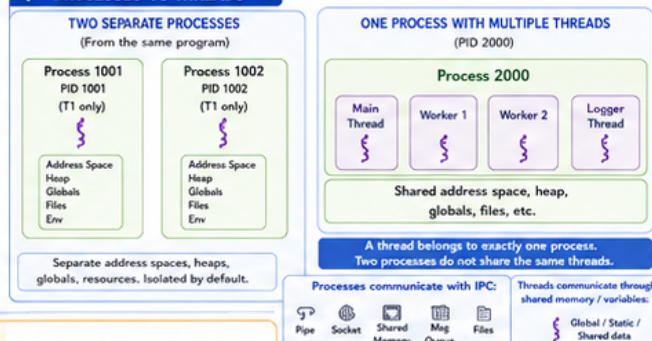
Concurrency = Multiple tasks making progress overlapping in time.
Parallelism = Tasks literally executing at the **SAME TIME** on different CPU cores.



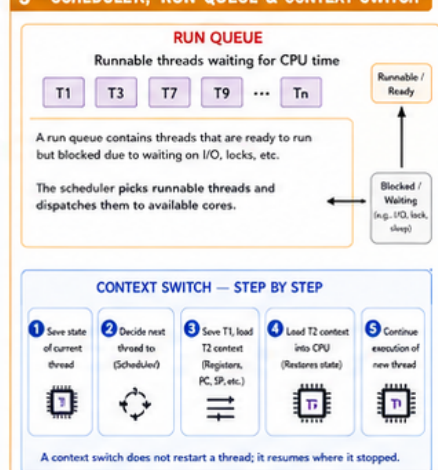
3 WHAT IS SHARED VS PRIVATE



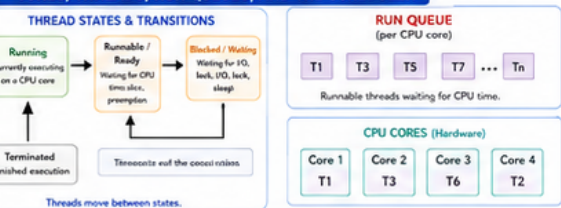
4 PROCESSES VS THREADS



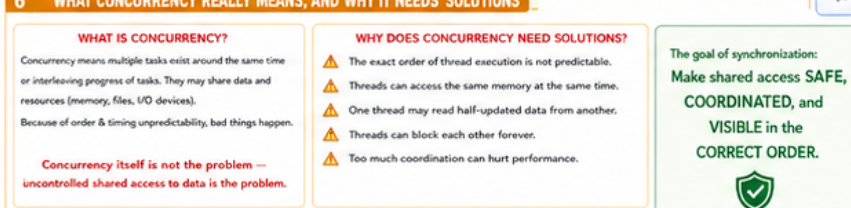
5 SCHEDULER, RUN QUEUE & CONTEXT SWITCH



5 CORES, THREADS, RUN QUEUES, AND THE SCHEDULER



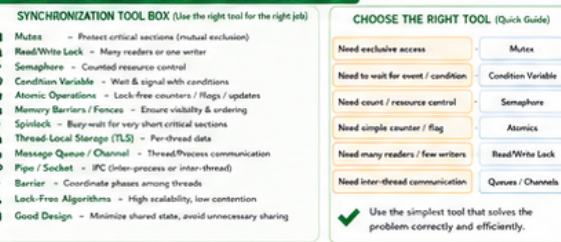
6 WHAT CONCURRENCY REALLY MEANS, AND WHY IT NEEDS SOLUTIONS



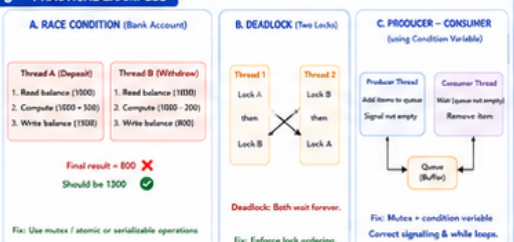
7 MAIN CONCURRENCY PROBLEMS

PROBLEM	WHAT IT IS	WHAT IT HAPPENS	WHAT GOES WRONG	EXAMPLE FIX
1 Race condition	Unsyncronized concurrent access to shared data	Threads access same data at the same time	Lost updates, corrupted data, inconsistent state	Mutex, atomic operations, locks, atomic sections
2 Deadlock	Each thread waits forever for another	Circular wait, hold & wait	Program hangs	Lock ordering, timeouts, try-locks, avoid cycles
3 Starvation	A thread gets no CPU time or resources	No fairness, others always win	Thread never runs, starves	Fair scheduling, priorities, round-robin
4 Livelock	Threads keep reacting to each other, no progress	Quickly retry operations, conflicting actions	High CPU use, no progress	Backoff, random delays, change strategy
5 Visibility / Ordering	Writes not visible to other threads in order	CPU caches, compiler reordering, weak memory	Stale reads, incorrect behavior	Memory barriers, atomic, volatile, fences
6 Lost updates	One thread overwrites another's update	Read-modify-write race	Data loss, incorrect results	Locks, atomic, RMW, transactions
7 False sharing	Threads modify different vars on same cache line	Invalidation ping-pong	Poor performance, scalability loss	Padding, align data, reduce sharing
8 Priority inversion	Low-priority thread blocks high-priority thread	Mutex held by low-priority thread	High priority waits unnecessarily	Priority inheritance, ceiling protocol
9 Lost wakeup / Missed signal	Signal/condition sent before thread waits	Bad timing of signal vs wait	Thread waits forever ("sleep forever")	Use condition variables correctly

8 GENERAL SOLUTIONS AND WHEN TO USE THEM



9 PRACTICAL EXAMPLES



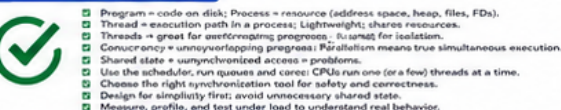
10 THREADS VS PROCESSES: WHEN TO USE WHICH



11 PERFORMANCE TERMS BRIEFLY EXPLAINED



12 FINAL TAKEAWAYS



11 FINAL TAKEAWAYS

- Program = code on disk; Process = resource (address space, heap, files, FDs).
- Thread = execution path in a process; Lightweight; shares resources.
- Threads = great for performance; processes = great for isolation.
- Concurrency means overlapping progress; parallelism means true simultaneous execution.
- Shared state = unsyncronized access = problems.
- Use the scheduler, run queues and cores: CPUs run one (or a few) threads at a time.
- Choose the right synchronization tool for safety and correctness.
- Design for simplicity first; avoid unnecessary shared state.
- Measure, profile, and test under load to understand real behavior.